

torch.functional.unique#

<https://pytorch.org/docs/stable/generated/torch.functional.unique.html>

Description:

Returns the unique elements of the input tensor. This function is different from `torch.unique_consecutive()` in the sense that this function also eliminates non-consecutive duplicate values. Currently in the CUDA implementation and the CPU implementation, `torch.unique` always sort the tensor at the beginning regardless of the `sorted` argument. Sorting could be slow, so if your input tensor is already sorted, it is recommended to use `torch.unique_consecutive()` which avoids the sorting.

Function Signature

```
torch.functional.unique(input, sorted=True,  
return_inverse=False, return_counts=False, dim=None) →  
tuple[Tensor, Tensor, Tensor][source]#
```

Parameters

Returns

Return type

Returns:

(Tensor, Tensor (optional), Tensor (optional))

Code Examples

```
>>> output = torch.unique(torch.tensor([1, 3, 2, 3],
dtype=torch.long))
>>> output
tensor([1, 2, 3])

>>> output, inverse_indices = torch.unique(
...     torch.tensor([1, 3, 2, 3], dtype=torch.long),
sorted=True, return_inverse=True)
>>> output
tensor([1, 2, 3])
>>> inverse_indices
tensor([0, 2, 1, 2])

>>> output, inverse_indices = torch.unique(
...     torch.tensor([[1, 3], [2, 3]], dtype=torch.long),
sorted=True, return_inverse=True)
>>> output
tensor([1, 2, 3])
>>> inverse_indices
tensor([[0, 2],
        [1, 2]])

>>> a = torch.tensor([
...     [
...         [1, 1, 0, 0],
...         [1, 1, 0, 0],
...         [0, 0, 1, 1],
```

```

...     ],
...     [
...         [0, 0, 1, 1],
...         [0, 0, 1, 1],
...         [1, 1, 1, 1],
...     ],
...     [
...         [1, 1, 0, 0],
...         [1, 1, 0, 0],
...         [0, 0, 1, 1],
...     ],
... ])

```

```

>>> # If we call `torch.unique(a, dim=0)`, each of the tensors
`a[idx, :, :]`

```

```

>>> # will be compared. We can see that `a[0, :, :]` and `a[2,
:, :]` match

```

```

>>> # each other, so one of them will be removed.

```

```

>>> (a[0, :, :] == a[2, :, :]).all()
tensor(True)

```

```

>>> a_unique_dim0 = torch.unique(a, dim=0)

```

```

>>> a_unique_dim0
tensor([[[0, 0, 1, 1],
        [0, 0, 1, 1],
        [1, 1, 1, 1]],
       [[1, 1, 0, 0],
        [1, 1, 0, 0],
        [0, 0, 1, 1]]])

```

```

>>> # Notice which sub-tensors from `a` match with the sub-
tensors from

```

```

>>> # `a_unique_dim0`:

```

```

>>> (a_unique_dim0[0, :, :] == a[1, :, :]).all()
tensor(True)

```

```

>>> (a_unique_dim0[1, :, :] == a[0, :, :]).all()
tensor(True)

```

```

>>> # For `torch.unique(a, dim=1)`, each of the tensors `a[:,

```

```

idx, :]' are
>>> # compared. `a[:, 0, :]' and `a[:, 1, :]' match each other,
so one of
>>> # them will be removed.
>>> (a[:, 0, :] == a[:, 1, :]).all()
tensor(True)
>>> torch.unique(a, dim=1)
tensor([[[0, 0, 1, 1],
        [1, 1, 0, 0]],
        [[1, 1, 1, 1],
        [0, 0, 1, 1]],
        [[0, 0, 1, 1],
        [1, 1, 0, 0]]])

>>> # For `torch.unique(a, dim=2)`, the tensors `a[:, :, idx]`
are compared.
>>> # `a[:, :, 0]` and `a[:, :, 1]` match each other. Also,
`a[:, :, 2]` and
>>> # `a[:, :, 3]` match each other as well. So in this case,
two of the
>>> # sub-tensors will be removed.
>>> (a[:, :, 0] == a[:, :, 1]).all()
tensor(True)
>>> (a[:, :, 2] == a[:, :, 3]).all()
tensor(True)
>>> torch.unique(a, dim=2)
tensor([[[0, 1],
        [0, 1],
        [1, 0]],
        [[1, 0],
        [1, 0],
        [1, 1]],
        [[0, 1],
        [0, 1],
        [1, 0]]])

```

Notes & Warnings

NOTE

Note This function is different from `torch.unique_consecutive()` in the sense that this function also eliminates non-consecutive duplicate values.

NOTE

Note Currently in the CUDA implementation and the CPU implementation, `torch.unique` always sort the tensor at the beginning regardless of the sort argument. Sorting could be slow, so if your input tensor is already sorted, it is recommended to use `torch.unique_consecutive()` which avoids the sorting.

Detailed Documentation

Documentation

Returns the unique elements of the input tensor.

This function is different from `torch.unique_consecutive()` in the sense that this function also eliminates non-consecutive duplicate values.

Currently in the CUDA implementation and the CPU implementation, `torch.unique` always sort the tensor at the beginning regardless of the sort argument. Sorting could be slow, so if your input tensor is already sorted, it is recommended to use `torch.unique_consecutive()` which avoids the sorting.

input (Tensor) – the input tensor

sorted (bool) – Whether to sort the unique elements in ascending order before returning as output.

return_inverse (bool) – Whether to also return the indices for where elements in the original input ended up in the returned unique list.

return_counts (bool) – Whether to also return the counts for each unique element.

dim (int, optional) – the dimension to operate upon. If None, the unique of the flattened input is returned. Otherwise, each of the tensors indexed by the given dimension is treated as one of the elements to apply the unique operation upon. See examples for more details. Default: None

A tensor or a tuple of tensors containing

output (Tensor): the output list of unique scalar elements.

inverse_indices (Tensor): (optional) if return_inverse is True, there will be an additional returned tensor (same shape as input) representing the indices for where elements in the original input map to in the output; otherwise, this function will only return a single tensor.

counts (Tensor): (optional) if return_counts is True, there will be an additional returned tensor (same shape as output or output.size(dim), if dim was specified) representing the number of occurrences for each unique value or tensor.

(Tensor, Tensor (optional), Tensor (optional))

